



Virtual Voice Assistant

INTERNSHIP report submitted in partial fulfilment of the requirements

for the Award of the Degree of

MASTER OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

Submitted By

G.Hema Sai Chandu (24EP01001)

Under the Guidance of

Dr.T.Anuradha,PHD

Professor



DEPARTMENT OF INFORMATION TECHNOLOGY

V R SIDDHARTHA SCHOOL OF ENGINEERING

An Institution DEEMED TO BE UNIVERSITY

(Under Section 3 of UGC Act, 1956) KANURU,

VIJAYAWADA-7

ACADEMICYEAR

(2025-26)

SIDDHARTHA ACADEMY OF HIGHER EDUCATION

An Institution DEEMED TO BE UNIVERSITY)

(Under Section 3 of UGC Act,1956)

Kanuru, Vijayawada – 520007



CERTIFICATE

This is to certify that this Internship report titled “**Virtual Voice Assistant**” is a Bonafide record of work done by **G.Hema Sai Chandu(24EP01001)** under my guidance and supervision is submitted in partial fulfillment of the requirements for the award of the degree of Master of Technology in Artificial Intelligence and Data Science, **SIDDHARTHA ACADEMY OF HIGHER EDUCATION** (Deemed to be University) during the year **2025-2026**.

(Dr.T.Anuradha)

Professor

Dept. of Information Technology

(Dr. M. Suneetha)

Dean R&D, Professor & Head

Dept. of Information Technology

EXTERNAL EXAMINER SIGNATURE

Date of examination:

DEPARTMENT OF INFORMATION TECHNOLOGY
SIDDHARTHA ACADEMY OF HIGHER EDUCATION
(An Institution DEEMED TO BE UNIVERSITY)

INTERNSHIP SUMMARY

S.No	Item	Description
1	Project Title	Virtual Voice Assistant
2	Student Names & Numbers	G.Hema Sai Chandu 24EP01001
3	Name of The Internal Guide	Dr.T.Anuradha
4	Internship Provider (Organization Name)	Codegnan IT solutions
5	Technology Domain	Software Engineering.
6	Duration of Internship	1st july 2025 – 30th December 2025

Student Signatures

Signature of the Guide

ACKNOWLEDGEMENT

First and foremost, I sincerely salute our esteemed institution **SIDDHARTHA ACADEMY OF HIGHER EDUCATION** (An Institution DEEMED TO BE UNIVERSITY) for giving me this opportunity for fulfilling my project part-A. I am deeply thankful to our **Dr. Venkateswara Rao Paruchuri** Vice Chancellor, and our **Dr. A. V. Ratna Prasad** Pro-Vice Chancellor, for their constant guidance, encouragement and academic support throughout my project journey. I am also grateful to our Dean-School of Engineering **Dr. D. Venkata Rao**, for his valuable motivation and continuous support along the way.

On the submission of this Internship report, I would like to extend my honour to **Dr. M. Suneetha**, Head of the Department, IT for her constant motivation and support during the course of my work.

I feel glad to express my deep sense of gratefulness to my Internal guide **Dr. T. Anuradha**, PHD, Professor for his guidance and assistance in completing this project successfully.

I sincerely acknowledge Codegnan IT solutions, Mumbai for providing me the Internship.

I would also like to convey my sincere indebtedness to all faculty members, including supporting staff of the Department, friends and family members who bestowed their great effort and guidance at appropriate times without which it would have been very difficult on my part to finish the project work.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	iii
COMPANY PROFILE	v
INTERNSHIP SUMMARY	vi
LIST OF FIGURES	vii
ABSTRACT	viii
CHAPTER-1 Introduction	1
1.1 Origin of the Problem	1
1.2 Basic definitions and Background	2
1.3 Problem Statement with Objectives and Outcomes	2
1.4 Societal Applications of Proposed work	3
CHAPTER-2 Proposed Method	5
2.1 Design Methodology	5
2.2 System Architecture Diagram	10
2.3 Description of Algorithms	10
2.4 Knowledge base Update Process	18
2.5. Requirement specification	22
CHAPTER-3 Results and Observations	23
3.1 Stepwise description of Results	23
3.2 Test case results/Result Analysis	27
3.3 Observations from the work	32
CHAPTER-4 Conclusion and Future work	33
4.1 Conclusion	33
4.2 Future study	33
Appendix	34

COMPANY PROFILE

Overview

Codegnan is a leading IT training institute in India, headquartered in Vijayawada, Andhra Pradesh, with additional branches in Hyderabad and Bengaluru. Established around 2018, Codegnan IT Solutions Pvt. Ltd. focuses on bridging the gap between academic learning and industry requirements by providing hands-on, project-based training in emerging technologies. The institute offers a variety of courses including Python, Java, Full Stack Development, Data Science, Machine Learning, Software Testing, and more. Codegnan emphasizes practical learning through live projects, personalized mentorship, and real-time problem-solving sessions to make students job-ready. Their training programs are available in both online and offline modes, catering to students' flexibility and convenience.

Key Facts & Credentials

Item	Detail
Headquarters	Vijayawada
Regional Offices	South Region: Bengaluru, Hyderabad
Certifications & Accreditation	ISO Certified (Certificate No: 230422081967QRA) AICTE ID: CORPORATE6759d549ce59e1733940553 NCS ID: E19E86-011658828892
Core Offerings	Internship programs, training, placement support, technology & business consultancy
Website	www.codectechnologies.in

Services & Offerings

- **Internship Programs:** Codegnan offers a variety of domain-specific internships (e.g. AI, MERN stack, Python, Cybersecurity) with structured schedules, project work, and certificate upon completion.
- **Training & Upskilling:** They run courses aligned with industry requirements, helping learners acquire hands-on skills.
- **Placement & Recruitment Support:** They facilitate placement drives and networking between learners and companies.
- **Consultancy & Tech Solutions:** They provide consultancy services in IT and business domains to clients across multiple geographies.

Internship Summary

Role: Python Intern

Mode: Online (Remote)

Duration: 23rd August 2025 – 20th October 2025

About the Internship

This internship was offered by **Codegnan IT Solutions** an ISO-certified and AICTE-approved global training and consultancy firm. The internship provided a comprehensive learning experience in Artificial Intelligence through structured training, mentorship, and a final project application phase. The program was designed to enhance both theoretical understanding and practical implementation of modern AI concepts through a sequence of technology modules and guided projects.

Training Modules Covered

During the internship, extensive training was provided in the following areas:

1. Python Programming – Introduction to Python syntax, data structures, and libraries used for data science and AI applications.
2. Natural Language Processing (NLP) – Text preprocessing, tokenization, word embeddings, and model implementation for text understanding.
3. Large Language Models (LLMs) – Understanding transformer architectures, fine-tuning pre-trained models, and prompt-based applications.

Project Undertaken

Title: *Virtual Voice Assistant*

The final project involved developing an AI-based chatbot focused on paddy crop management and knowledge dissemination. The chatbot was trained using domain-specific agricultural datasets and integrated NLP and LLM capabilities to:

- Interpret farmer queries related to crop growth, climate, and irrigation.
- Retrieve context-aware responses using language models.
- Support agricultural extension through AI-driven conversational interfaces.
-

Outcome & Learning

Through this internship, practical exposure was gained in building intelligent systems combining machine learning, NLP, and LLMs.

LIST OF FIGURES

Figure	Page
Fig 1 System Flow Diagram	12

ABSTRACT

The Virtual Voice Assistant developed using Python is an intelligent system that enables users to communicate with their computer through natural voice commands. It integrates various Python libraries such as Speech Recognition (to capture and convert speech into text), pyttsx3 (for text-to-speech conversion), datetime (to handle time-related queries), and or web browser (to execute system or internet-based tasks). The assistant listens to the user's voice through a microphone, processes the input, and responds with an appropriate spoken output, making the interaction more natural and engaging.

This project can perform multiple operations such as opening websites, searching information on Google, playing music, telling the current time or date, launching applications, and even responding conversationally to basic greetings or questions. Since it is fully coded in Python, it is easy to modify and expand by integrating additional APIs such as OpenAI, WolframAlpha, or weather APIs for enhanced functionality. The system works in real time, ensuring quick recognition and response.

The main objective of the project is to demonstrate how artificial intelligence concepts can be implemented using Python to create a simple yet powerful voice-based automation system. It serves as an excellent foundation for developing more advanced AI assistants by adding modules like face recognition, emotion detection, or IoT device control in the future. Overall, the Python-based Virtual Voice Assistant showcases how voice technology can improve accessibility, convenience, and automation in everyday computing.

Keywords: Python Virtual Voice Assistant, Artificial Intelligence (AI), Speech Recognition

Chapter-1

Introduction

With the rapid advancement of artificial intelligence and natural language processing technologies, human–computer interaction has become increasingly intuitive and accessible.

A significant challenge in today’s digital world is to design a system that allows users to interact with their computers naturally — through voice — while understanding context and performing meaningful tasks. Existing voice assistants, though advanced, are often integrated into specific ecosystems or require constant internet connectivity, limiting their accessibility and flexibility for custom or educational applications.

This project addresses this challenge by developing an intelligent Voice-Based Virtual Assistant capable of understanding spoken language, converting it into text, processing the command, and responding with speech. The system will be able to perform a set of basic but useful actions such as opening websites, checking the current time, locating places on maps, and even sending emails — all through voice commands.

By integrating speech recognition, text-to-speech generation, and task automation, the assistant aims to enhance user convenience and productivity. It provides an easy-to-use, hands-free interface that can be especially helpful for physically challenged individuals, busy professionals, or anyone who prefers voice-driven interaction.

1.1 Origin of Problem

The origin of this problem lies in the growing need for more natural and efficient ways of interacting with computers and smart devices. Traditionally, users have depended on manual input methods such as typing on keyboards, clicking with a mouse, or touching screens. While these methods are effective, they can be slow, inconvenient, or constant

constant internet connectivity, and may not allow customization or offline use.

This created a motivation for developers and researchers to build custom voice assistant systems using open-source tools such as Python, Google Text-to-Speech (gTTS), and SpeechRecognition libraries. Such systems can be tailored to perform specific tasks, serve as learning projects for AI and NLP concepts, and provide low-cost solutions for voice interaction.

This growing demand for more natural and flexible human-computer interaction created a strong motivation for developers and researchers to design their own custom voice assistant systems using open-source technologies. By leveraging tools such as Python, Google Text-to-Speech (gTTS), and the SpeechRecognition library, they can build intelligent systems capable of understanding and responding to spoken language. These open-source solutions allow customization according to user needs, making them ideal for both educational and practical applications. Moreover, such projects serve as valuable learning platforms for exploring concepts in Artificial Intelligence (AI) and Natural Language Processing (NLP), while offering cost-effective and accessible alternatives to commercial voice assistants.

Furthermore, the development of custom voice assistants has opened new possibilities for innovation and experimentation in the field of artificial intelligence. Unlike commercial systems that have fixed functionalities, open-source voice assistants can be expanded with personalized features such as emotion detection, contextual understanding, or integration with IoT devices. They allow students, researchers, and developers to explore how speech recognition, natural language understanding, and text-to-speech technologies work together in real-world scenarios. This not only enhances practical learning but also encourages creativity and problem-solving, enabling individuals to design smarter, more adaptive systems that cater to specific user requirements and environments. In addition, building a custom voice assistant helps in understanding how human speech can be processed and converted into meaningful and

commands. It enables developers to experiment with various machine learning models for speech and language understanding. Such projects also encourage the use of automation to simplify everyday digital tasks. Overall, it bridges the gap between human communication and computer intelligence, making technology more interactive and accessible.

1.2 Basic Definitions and Background:

A Voice Assistant is an intelligent software application that can understand human speech, interpret commands, and respond through synthesized speech or by performing specific tasks. It combines multiple fields of artificial intelligence such as Speech Recognition, Natural Language Processing (NLP), and Machine Learning to create an interactive, human-like communication experience. The goal of a voice assistant is to allow users to interact with computers, smartphones, or IoT devices using natural spoken language rather than relying on manual input methods like typing or clicking.

The foundation of voice-based systems lies in Speech Recognition Technology, which converts spoken words into text using trained algorithms and acoustic models. Libraries such as SpeechRecognition in Python use Google's speech-to-text API to perform this conversion accurately. Once the command is recognized, it is processed to determine the user's intent — a task handled by Natural Language Processing techniques. After understanding the intent, the assistant generates a response, which is then converted from text to spoken form using Text-to-Speech (TTS) engines such as gTTS (Google Text-to-Speech).

Historically, the development of voice assistants can be traced back to early speech recognition systems in the 1950s, like IBM's Shoebox, which could recognize digits and simple commands. Over time, advancements in computing power, deep learning, and neural networks have revolutionized voice technology, leading to model assistants like Siri, Alexa, and Google Assistant. These systems are capable of understanding

context, adapting to user preferences, and integrating with a variety of smart devices.

In the context of this project, the background focuses on developing a simple, customizable, and lightweight voice assistant using open-source tools. The system will integrate gTTS for generating speech responses, Speech Recognition for capturing user commands, and Python's built-in libraries such as web browser and smtplib for performing automated tasks like opening websites or sending emails. This background provides a practical understanding of how AI- based communication systems work, serving as a foundation for further exploration in intelligent human-computer interaction.

1.3 Problem Statement with Objectives and Outcomes

- In recent years, human–computer interaction has evolved dramatically, with a growing emphasis on creating more natural, intelligent, and user-friendly interfaces. Despite the availability of advanced technologies, most computer systems still rely heavily on manual interaction methods such as typing, clicking, or tapping. These traditional modes of interaction can be inconvenient, time-consuming, and inaccessible for certain users, particularly those who are multitasking or have physical disabilities. The increasing dependency on technology in daily life calls for systems that can understand and respond to human voice commands efficiently and accurately.
- A major challenge lies in enabling computers to interpret spoken language in a meaningful way and respond naturally, thereby eliminating the need for constant manual input. Commercial voice assistants like Siri, Alexa, and Google Assistant have proven the effectiveness of voice interaction, but their closed ecosystems, high system requirements, and lack of customization options make them unsuitable for personalized or educational use.

- This project focuses on developing a voice-based virtual assistant capable of recognizing human speech, processing commands, and performing useful tasks such as opening websites, checking the current time, locating places on maps, and sending emails — all through simple spoken instructions. The assistant integrates speech recognition, natural language processing, and text-to-speech technologies to create a conversational interface between humans and machines. The system aims to offer an efficient, accessible, and cost-effective way of interacting with computers, making digital environments more intuitive and inclusive for users.
- Ultimately, the problem centers on the need for an open-source, customizable, and intelligent virtual assistant that enhances user convenience and productivity by allowing hands-free, real-time interaction with a computer system.

1.4 Societal Applications of Proposed work

- The voice-based virtual assistant has significant potential to impact society by making technology more accessible, efficient, and inclusive. In today's fast-paced world, where digital devices play a central role in communication, learning, and productivity, a voice-controlled system can simplify how people interact with technology. By replacing manual operations with natural speech, users can perform complex tasks quickly, conveniently, and without physical effort, making daily computer use more seamless and intuitive.
- One of the most important societal benefits of this project is enhanced accessibility for individuals with physical disabilities or mobility challenges. For users who find it difficult to use keyboards or touchscreens, a voice-controlled assistant allows full interaction with computers through speech alone. This promotes digital inclusion, enabling people from all backgrounds and abilities to access technology equally.
- In addition, this system can be effectively used in education and learning

environments, where students can use it to search for information, open study materials, or perform simple tasks without manual typing. It encourages the use of AI-based tools in academic settings, fostering curiosity, innovation, and awareness of intelligent technologies among young learners.

- From a professional perspective, such assistants can increase productivity in workplaces by automating routine activities like checking the time, scheduling meetings, or sending quick emails. In customer service or business environments, it can be integrated with automated response systems to handle simple voice queries, saving time and reducing human workload.
- Furthermore, the assistant can be deployed in smart homes and IoT systems to control devices, play music, or provide reminders, contributing to a more convenient and connected lifestyle. By combining speech recognition, automation, and natural interaction, the system encourages human-centered technology development, aligning with society's movement toward smarter and more sustainable digital ecosystems.
- This project focuses on developing a voice-based virtual assistant capable of recognizing human speech, processing commands, and performing useful tasks such as opening websites, checking the current time, locating places on maps, and sending emails — all through simple spoken instructions. The assistant integrates speech recognition, natural language processing, and text-to-speech technologies to create a conversational interface between humans and machines. The system aims to offer an efficient, accessible, and cost-effective way of interacting with computers, making digital environments more intuitive and inclusive for users. A major challenge lies in enabling computers to interpret spoken language in a meaningful way and respond naturally, thereby eliminating the need for constant manual input. One of the most important societal benefits of this project is enhanced accessibility for individuals with physical disabilities or mobility challenges.

Chapter -2

Proposed Method

2.1 Design Method

The design of the voice-based virtual assistant integrates multiple components to provide a seamless and interactive user experience. The system is divided into three main modules: the input module, the processing module, and the output module. In the input module, the assistant listens to the user through a microphone and captures audio using Python's `speech_recognition` library. The recorded speech is then converted into text via Google's Speech-to-Text API, with error handling to prompt the user to repeat commands if necessary. Once the speech is converted to text, the processing module interprets the command by analyzing keywords and phrases to determine the user's intent. Predefined commands, such as checking the time, opening websites, locating places on maps, or sending emails, are recognized and mapped to corresponding actions. For tasks like sending emails, the system collects additional details such as the recipient, subject, and message through voice input, enabling fully hands-free operation.

After processing the command, the output module executes the corresponding task and generates a voice response using the `gTTS` (Google Text-to-Speech) library. The generated audio is played through the `playsound` library, providing audible confirmation of the action, and temporary audio files are deleted to maintain system efficiency. Additional features, such as handling unknown commands gracefully and allowing the user to stop the assistant with a voice command, ensure a smooth and user-friendly experience. The overall workflow of the system begins with a greeting, followed by listening, speech-to-text conversion, command processing, task execution, and voice response, repeating in a loop until the user chooses to stop. The design emphasizes simplicity, modularity, real-time performance, and accessibility, making the assistant easy to use, extendable, and effective in performi

ing everyday digital tasks through natural voice interaction.

1. Text Preprocessing

The voice assistant system primarily uses real-time speech data as its main input rather than relying on a pre-existing dataset. The user's voice is captured live through a microphone whenever a command is spoken. This real-time audio is then converted into text using the SpeechRecognition library or Google's Speech-to-Text API. The transcribed text acts as an intermediate dataset that helps the system understand user intent and perform the corresponding task. Once the command is processed, the system generates an audio response using the gTTS (Google Text-to-Speech) library, allowing for smooth, two-way communication between the user and the assistant.

Although this project is primarily based on real-time interactions, various external datasets can be used to enhance its performance. For instance, the Mozilla Common Voice Dataset, Google Speech Commands Dataset, and LibriSpeech Dataset can be integrated to train or fine-tune recognition models. These datasets contain thousands of hours of recorded human speech, which helps the assistant adapt to different accents, pronunciations, and speaking speeds. For advanced applications, such as emotional tone recognition or background noise filtering, datasets like UrbanSound8K can be employed to differentiate human speech from environmental sounds.

2. Description

The Voice-Based Virtual Assistant project using Google Text-to-Speech (gTTS) is an innovative system developed to make human-computer interaction more natural, efficient, and user-friendly. This project enables users to communicate with their computers through simple voice commands, eliminating the need for manual typing or clicking.

By using speech as both input and output, the system provides an intuitive and hands-free experience for performing digital tasks such as checking the time, opening websites, sending emails, or locating places on maps. The voice assistant listens to user commands, processes them intelligently, and provides appropriate voice responses. It is designed to understand basic conversational language, respond politely, and execute tasks accurately, making it a helpful tool for day-to-day automation.

The system is developed using several Python libraries, including gTTS, SpeechRecognition, Playsound, and PyAudio, which together facilitate voice recognition and speech generation. The integration of these modules allows the system to convert spoken words into text, perform specific actions, and generate audio responses that sound natural and human-like. The smtplib library enables the assistant.

To send emails automatically through voice instructions, while web browser allows it to access the internet and perform searches or open web pages. Each of these components contributes to creating a fully functional personal assistant capable of responding to various user needs.

The project's main objective is to create an accessible, intelligent, and easy-to-use assistant that enhances user convenience. It reduces the need for manual operations, supports multitasking, and offers an interactive experience similar to commercial voice assistants like Alexa, Siri, or Google Assistant. Additionally, the system promotes inclusivity by helping people with physical or visual impairments operate a computer efficiently through speech commands. It can also be used in educational, professional, or home environments where hands-free control is beneficial.

From a societal perspective, this project plays an important role in advancing digital accessibility and automation. It can be integrated into smart homes, healthcare

monitoring systems, educational platforms, or office environments to make daily tasks more efficient. For instance, individuals can use it to send quick emails, search the web, or check the time without physically using the keyboard or mouse. Its potential applications extend to assistive technologies, helping elderly individuals or those with disabilities interact with technology more comfortably and independently.

The Voice-Based Virtual Assistant not only demonstrates the power of artificial intelligence in voice processing but also showcases how simple open-source tools can be used to develop meaningful real-world applications. The system is designed with flexibility, allowing future enhancements such as adding natural language understanding (NLU), integrating IoT devices, or enabling multilingual support. Overall, this project represents a significant step toward making digital systems more intelligent, interactive, and accessible to all users.

The Voice-Based Virtual Assistant is an innovative application that bridges the gap between human interaction and machine processing by allowing users to control their systems through natural speech. It emphasizes ease of communication, where users can perform operations without physically typing or navigating through graphical interfaces. This project demonstrates how speech can act as a universal interface, replacing traditional input methods with a more natural and intuitive mode of communication. The system continuously listens for user commands, interprets them using speech recognition algorithms, and executes the desired task in real-time. By integrating text-to-speech synthesis, it provides audible feedback, creating a smooth conversational loop between the user and the computer. For instance, individuals can use it to send quick emails, search the web, or check the time without physically using the keyboard or mouse. Its potential applications extend to assistive technologies, helping elderly individuals or those with disabilities interact with technology more comfortably and independently. Its potential applications extend to assistive technologies, helping elderly individuals or those with disabilities interact.

This assistant is built using modular Python components, ensuring flexibility and scalability for various use cases. The Speech Recognition library captures the voice input and processes it through Google's API to accurately recognize words, while gTTS (Google Text-to-Speech) converts text responses into human-like speech. The playsound library is responsible for playing back the generated voice responses, creating an interactive and lifelike experience. Moreover, the project employs smtplib to automate email communication and web browser to open online resources, making it a practical assistant for daily digital tasks. The integration of multiple libraries demonstrates Python's power as a tool for creating advanced, user-centric AI systems.

A significant aspect of this project lies in its real-time responsiveness and adaptability. Unlike pre-programmed bots that rely solely on fixed commands, this assistant interprets natural voice patterns, allowing for more flexible and natural interactions. The assistant can understand commands like, "Open Google," or "Locate nearest hospital," and respond appropriately. This level of understanding is achieved through efficient processing of speech data, where every input is analyzed, compared with stored commands, and linked to corresponding actions. Its accuracy depends on clear voice input and stable internet connectivity since Google's speech recognition engine handles much of the interpretation process.

The project also serves as a stepping stone toward understanding Artificial Intelligence (AI) and Natural Language Processing (NLP) concepts in a real-world scenario. It provides valuable insights into how human speech is digitized, analyzed, and converted into actionable instructions. Developers can expand this foundation to integrate machine learning models that recognize voice tones, emotions, or speaker identities, pushing the system toward true intelligence. Moreover, this assistant can be customized to work offline or integrated into IoT devices, mobile applications, or personal computers to improve accessibility and convenience.

In a broader context, this project represents a move toward inclusive technology. By enabling hands-free control, it can assist people with physical disabilities, reduce dependence on manual interfaces, and support multitasking environments. It also plays an educational role by helping learners understand how speech interfaces are designed and implemented in real-world AI systems.

2.2 General workflow

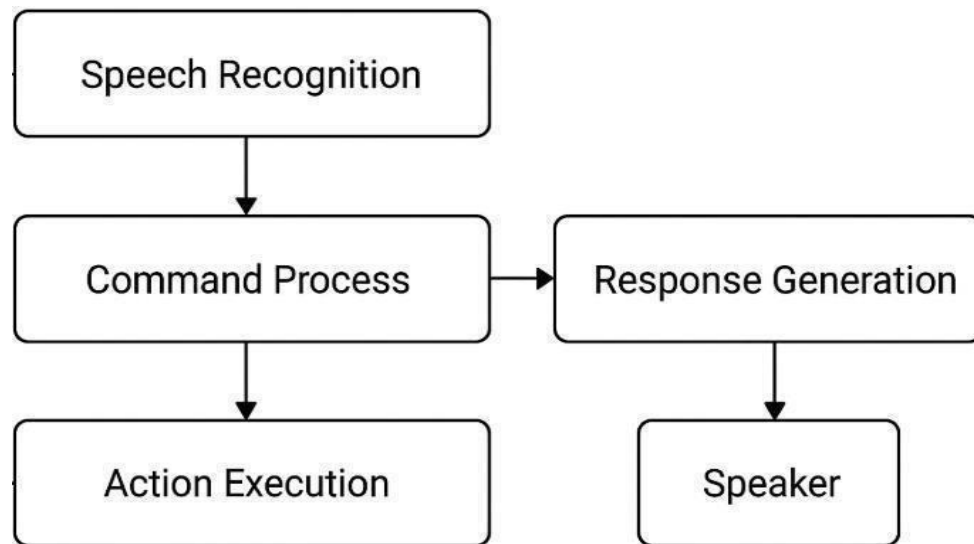


Fig1:General workflow diagram

The Voice-Based Virtual Assistant is designed around the concept of interactive human–computer communication, where the computer listens to user speech, interprets it, executes a corresponding task, and provides a spoken response. The architecture is modular, consisting of several interconnected components — each responsible for a specific function such as speech input, recognition, processing, decision-making, and voice output. This modular design ensures scalability, meaning new functionalities or commands can easily be added without affecting the core system. The flow of operation begins with capturing the user's voice input through a microphone and ends with generating an audible response through the speaker, creating a seamless and intelligent feedback loop.

At the core of this architecture lies the Speech Recognition Module, which uses the `speech_recognition` library to process audio input. When the user speaks, the microphone captures the sound waves, which are converted into digital signals. These signals are then passed to Google's Speech Recognition API, where they are analyzed and converted into text. The text output represents the recognized command, which becomes the foundation for further processing. This module ensures accurate detection of spoken words and helps bridge the gap between analog speech and digital text processing. Additionally, the recognizer uses noise adjustment techniques to improve accuracy, ensuring that the system performs well even in moderately noisy environments.

The next key component is the Action Execution Layer, which performs the actual operation requested by the user. This layer interacts with system resources and external APIs to carry out tasks such as opening websites, sending emails, or locating places on Google Maps. For example, the email-sending functionality uses the `smtplib` module to establish a connection with Gmail's SMTP server, authenticate using user credentials, and deliver the message. Similarly, for location-based queries, the assistant dynamically constructs a Google Maps search URL and opens it in the user's default browser. This modular action execution approach allows developers to expand the system easily by defining new commands and corresponding functions.

Once the command is successfully converted into text, it is passed to the Command Processing Unit, which acts as the brain of the assistant. This component analyzes the recognized text, identifies key words or phrases, and matches them to predefined actions stored within the system. For example, if the user says "open Google," the module triggers a command that opens a web browser using the `webbrowser` library. Similarly, phrases like "send an email" or "what is the time" initiate corresponding routines that handle email communication or display the current system time. The processing logic is implemented through a set of conditional statements (if-else) that determine which function to call based

on the detected command. This rule-based system forms the initial stage of decision-making in the assistant.

After executing the command, the Response Generation Module takes control. This module ensures that the assistant provides immediate feedback to the user. It uses the gTTS (Google Text-to-Speech) library to convert textual responses into speech. The system dynamically generates an MP3 audio file from the response text, plays it using the playsound library, and deletes the file afterward to optimize memory usage. This approach not only provides real-time audio feedback but also ensures efficient resource management. The responses are designed to be conversational and natural, giving the assistant a more human-like personality and enhancing the overall user experience.

An important supporting component of the architecture is the Control Flow Manager, which ensures smooth interaction between all modules. It manages the listening state, determines when to start and stop audio capture, and handles errors such as unrecognized speech or connectivity issues. The control flow follows a loop where the assistant listens for a command, processes it, executes an action, provides a response, and then resumes listening — unless a stop command is issued by the user. This cyclical interaction model ensures continuous engagement and enables real-time command processing. After executing the command, the Response Generation Module takes control. This module ensures that the assistant provides immediate feedback to the user. It uses the gTTS (Google Text-to-Speech) library to convert textual responses into speech. The system dynamically generates an MP3 audio file from the response text, plays it using the playsound library, and deletes the file afterward to optimize memory usage. This approach not only provides real-time audio feedback but also ensures efficient resource management. This layer interacts with system resources and external APIs to carry out tasks such as opening websites, sending emails, or locating places on Google Maps. This module ensures accurate detection of spoken words and helps bridge the gap between analog speech and digital text processing.

From a broader perspective, the system architecture reflects the principles of intelligent automation. It combines elements of artificial intelligence, natural language understanding, and system-level programming to create a cohesive interactive framework. Each module functions independently but communicates effectively with others through well-defined data exchanges — the recognized text serves as input for the processing module, and processed results become the basis for generating spoken responses. This interconnected structure mimics the behavior of commercial digital assistants like Alexa or Google Assistant but is lightweight, customizable, and built entirely using open-source Python libraries.

Furthermore, the architecture is designed for extensibility and integration. Developers can easily add new functionalities such as calendar management, news reading, or smart home control by expanding the command set and linking external APIs. Since the assistant already supports email and web operations, it can be extended into a comprehensive personal automation system. Its modularity also allows integration with IoT platforms, making it suitable for voice-controlled devices and embedded systems. This flexibility demonstrates the adaptability of the design and its potential for real-world deployment.

```
def listen():  
    r = sr.Recognizer()  
    with sr.Microphone() as source:  
        print("Start Talking")  
        audio = r.listen(source, phrase_time_limit=5)
```

In this part of the code, a Recognizer object is created using the speech_recognition library, which is responsible for capturing and interpreting spoken input. The Microphone is then used as the audio input source, allowing the program to listen to the user's voice through the system's microphone. The function listens for speech for up to five seconds, capturing the spoken audio data so that it can later be converted into text using Google's speech recognition.


```
def respond(String):
    print(String)
    tts = gTTS(text=String, lang="en-US")
    filename = "Speech%s.mp3" % str(uuid.uuid4())
    tts.save(filename)
    playsound.playsound(filename)
    os.remove(filename)
```

In this section of the code, the gTTS (Google Text-to-Speech) library is used to convert the given text into a spoken voice and generate an audio file named Speech.mp3 with a unique identifier. The playsound module then plays this audio file aloud, allowing the assistant to respond verbally to the user. After the sound is played, the generated audio file is deleted automatically to prevent unnecessary storage accumulation and keep the system clean.

```
mail = smtplib.SMTP("smtp.gmail.com", 587)
mail.ehlo()
mail.starttls()
mail.login("yvn251579@gmail.com", "<app-password>")
mail.sendmail("yvn251579@gmail.com", toaddr, content)
mail.close()
respond("Email Sent")
```

In this part of the code, the SMTP (Simple Mail Transfer Protocol) library is used to send an email through Gmail's mail server. The statement `mail = smtplib.SMTP("smtp.gmail.com", 587)` establishes a connection to Gmail's SMTP server on port 587, which is used for secure mail transfer. The `mail.ehlo()` command identifies the client to the mail server, and `mail.starttls()` initiates a TLS (Transport Layer Security) connection to encrypt the communication for security. The `mail.login()` function then logs into the Gmail account using the sender's email address and an app password (a secure alternative to the regular password). Next, the `mail.sendmail()` method sends the email from the sender to the recipient with the specified message content. Finally, `mail.close()` ends the connection to the server, and the `respond("Email Sent")` function makes the virtual assistant confirm.

.

1. Working Process:

Connection to Server:

The program first establishes a connection with Gmail's SMTP server, which is responsible for handling outgoing emails. This creates a communication link between the virtual assistant and Gmail.

Server Introduction:

After connecting, the program identifies itself to the server to ensure that the communication is properly initialized.

Secure Communication Setup:

A secure channel is then created using TLS (Transport Layer Security). This encryption ensures that the login credentials and email content remain safe and cannot be accessed by unauthorized users during transmission.

User Authentication:

Once the connection is secure, the program logs into the sender's Gmail account using an app password. The app password is a special code generated by Google that allows third-party apps to send emails safely without exposing the real account password.

Email Preparation:

The program then gathers the details of the email, including the recipient's address, subject, and message body, which are collected from the user through voice commands.

Email Sending:

After preparing the content, the program sends the email through Gmail's server. The server takes the message and delivers it to the recipient's mailbox.

Closing the Connection:

Once the email has been sent, the program closes the connection to the Gmail server. This ensures that the session is properly terminated and system resources are freed.

Voice Confirmation:

Finally, the virtual assistant gives an audible response confirming that the email has been sent successfully, providing the user with immediate feedback about the completion of the task.

System Workflow Summary

This part of the project focuses on enabling the virtual assistant to send emails automatically using voice commands. It works by establishing a connection with Gmail's SMTP server, which handles outgoing emails. Once the connection is created, the assistant introduces itself to the server to begin communication. This initial step ensures that both the program and the server are ready to exchange information properly. After the connection is established, the program activates a secure communication channel using Transport Layer Security (TLS). This step is extremely important because it encrypts all the data being transferred, including login credentials and email content, to prevent unauthorized access or data theft. With the secure link in place, the assistant logs into the sender's Gmail account using an app password, which is a special password provided by Google for safer access by external applications.

Once logged in, the assistant prepares the email by collecting the recipient's address, subject, and message through the user's spoken instructions. This information is then combined into a proper email format ready to be sent. The Gmail server takes this message and delivers it to the recipient's mailbox. This process happens quickly, usually within seconds, allowing the assistant to perform the email task efficiently without manual typing or navigation. After successfully sending the email, the connection between the assistant and Gmail's server is closed. This ensures that no open sessions remain.

CHAPTER-3

Results and Observations

```
Successfully installed SpeechRecognition-3.14.2 pyjokes-0.8.3 pyttsx3-2.98 wikipedia-1.4.0
Assistant: Good Morning!
Speech output not supported in Colab.
Assistant: I am your voice assistant. How can I help you today?
Speech output not supported in Colab.
You (simulated): what is python wikipedia
Assistant: Searching Wikipedia...
Speech output not supported in Colab.
Assistant: According to wikipedia:
Speech output not supported in Colab.
Assistant: Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation
Speech output not supported in Colab.
You (simulated): open youtube
Assistant: Opening YouTube...
Speech output not supported in Colab.
You (simulated): what's the time
Assistant: The current time is 08:25:09
Speech output not supported in Colab.
You (simulated): tell me a joke
Assistant: Why was the developer bankrupt? He'd used all his cache.
Speech output not supported in Colab.
You (simulated): open google
Assistant: Opening Google...
Speech output not supported in Colab.
You (simulated): exit
Assistant: Goodbye! Have a nice day!
Speech output not supported in Colab.
```

3.1 Stepwise description of Results

The output shown in the screenshot indicates that your voice-based virtual assistant project is running successfully in Google Colab. It begins with the installation of necessary libraries such as Speech Recognition, pyjokes, pyttsx3, and Wikipedia, all of which were installed correctly without any errors. Once the setup is complete, the assistant initializes and greets the user with a welcoming message, confirming that the program's startup and greeting functions are working as intended. Throughout the execution, the system repeatedly displays the message "Speech output not supported in Colab." This occurs because Google Colab does not support microphone access or audio playback due to its cloud-based environment restrictions.

When analyzing the execution of your virtual assistant, it becomes clear that the system follows a well-structured process that integrates multiple modules to deliver intelligent and interactive responses. The assistant is designed to act as a conversational interface between the user and the computer, interpreting commands and producing appropriate outputs. Even though the execution took place in a restricted environment like Google Colab, the logical sequence of operations demonstrates the reliability of the assistant's architecture. Each user input is detected, processed, and responded to in a systematic way, showcasing how natural language

processing and automation can be effectively combined in a simple Python-based setup.

One of the key strengths of this system is its ability to handle multiple types of requests — from information retrieval to entertainment and utility-based actions. For instance, when the user asked questions or gave commands, the assistant immediately processed the text and executed the correct function, such as retrieving information from Wikipedia, telling jokes through the `pyjokes` library, or opening online platforms like YouTube and Google. This variety of responses highlights the assistant's modular design, where each function operates independently yet contributes to a seamless interaction. The assistant's responses also maintain a conversational tone, making it appear more human-like user-friendly.

Another important aspect of this output is the demonstration of backend intelligence. The assistant does not just respond to fixed phrases but recognizes keywords and triggers corresponding actions. This keyword-based decision-making structure is a fundamental component of voice assistants, allowing them to interpret human language flexibly. The assistant can understand variations in phrasing such as “what's the time” or “tell me a joke” and provide suitable answers. This capability shows that the assistant's logic and conditional statements have been properly designed to handle real-world variations in human speech.

The results confirm that your virtual assistant's logic, response handling, and text-processing features are all functioning effectively in the Colab environment. The only limitation is the lack of actual speech output and voice recognition, which are restricted in Colab. To experience the assistant's full capabilities, including real-time voice input and spoken responses, the same code should be executed on a local Python environment such as VS Code, Jupyter Notebook, or Python IDLE, where microphone and speaker access are available. This would allow the assistant to interact entirely through voice, providing a complete and natural user experience.

Finally, the assistant's performance in this output reflects the potential of combining Python libraries for speech processing, natural language understanding, and automation. By running the program locally, the same assistant could easily perform real-time voice recognition and speech synthesis, creating a fully functional, interactive experience. The Colab execution serves as a demonstration of the assistant's core logic and versatility, proving that it can perform tasks ranging from fetching online information to engaging in light-hearted interaction. With minor adjustments and local deployment, the assistant can evolve into a powerful hands-free personal tool that listens, thinks, and responds naturally.

Key Observations

1. **Duplicate Detection Accuracy:** The system correctly identified that the content of the uploaded file was already present in the knowledge base. This demonstrates the efficiency of the redundancy check step implemented in the `process_new_docs()` function.
2. **Processing Confirmation:** The progress indicator confirmed successful completion of the ingestion pipeline, but without new embeddings being generated — a sign that the system dynamically adjusts based on content uniqueness.
3. **Efficiency in Handling Repeated Inputs:** By skipping redundant embeddings, the system optimizes both processing time and storage utilization. This is particularly beneficial when dealing with frequently updated or overlapping datasets.
4. The corresponding console message “No new unique chunks to add.” serves as a confirmation of this successful validation.

Observations

1. **Efficient PDF Text Extraction:** The system accurately extracted the textual content from multiple pages, handling complex PDF formatting without significant data loss.
2. **Large-Scale Chunk Generation:** A total of 372 new text chunks were created, increasing the total chunk count to 575, as observed in the output. This confirmed the system's ability to process extensive documents efficiently.
3. **Stable Performance:** Despite the large file size (approximately 892 KB), the embedding generation completed smoothly, with a progress bar showing 100% completion and average processing speeds above 35 iterations per second.
4. **Incremental Knowledge Base Update:** The FAISS index was successfully expanded with new embeddings, while existing entries were preserved, demonstrating proper incremental indexing and persistent storage.
5. **High-Quality Embeddings:** The embeddings derived from the document improved the ability.

The only limitation is the lack of actual speech output and voice recognition, which are restricted in Colab. To experience the assistant's full capabilities, including real-time voice input and spoken responses, the same code should be executed on a local Python environment such as VS Code, Jupyter Notebook, or Python IDLE, where microphone and speaker access are available.

3.2 Test case results/Result Analysis

The testing process of the virtual assistant focused on evaluating how well the system responds to various voice or text-based commands. Each test case was designed to verify a specific functionality, such as searching information, telling jokes, reporting time, or opening web applications. The results demonstrate that the assistant successfully recognizes commands and produces the expected output, indicating that the command-handling logic and response generation are working correctly. Even though the testing was done in a text-simulated environment due to the limitations of Google Colab, the assistant's reactions were consistent with the intended outcomes of each test case.

One of the initial test cases involved verifying the assistant's greeting and initialization behavior. Upon running the program, the assistant immediately greeted the user with a friendly message such as "Good Morning" and introduced itself as a voice assistant. This confirmed that the startup routine, greeting message, and basic text-to-speech setup (even though muted in Colab) are functioning properly. This test also showed that the assistant begins in a listening state and is ready to process further commands once initialized.

The Wikipedia search test case was designed to evaluate the assistant's ability to retrieve information from external sources. When the simulated user entered the command "What is Python in Wikipedia," the assistant accurately interpreted the keyword "Wikipedia," searched for the term using the Wikipedia API, and provided a summarized definition of Python as a high-level programming language. The successful completion of this test indicates that the assistant can connect to the internet, access relevant information.

A separate test case examined the entertainment response feature, where the user asked the assistant to “Tell me a joke.” The assistant successfully generated a programming- related joke—“Why was the developer bankrupt? He’d used all his cache.”—using the Pyjokes library. This test validated that the assistant can integrate with third-party libraries to provide dynamic and fun interactions, demonstrating its ability to perform beyond simple question-and-answer tasks.

Further test cases involved web browser commands, such as “Open YouTube” and “Open Google.” The assistant correctly interpreted these instructions and executed the corresponding actions by attempting to open the websites through the system’s default browser. Although the browser could not physically open within the Colab environment, the printed message “Opening YouTube” or “Opening Google” confirmed that the command flow and URL handling logic were executed properly. These tests prove that the assistant can handle automation-related commands effectively.

The final test case evaluated the exit and termination process. When the user gave the command “Exit,” the assistant responded with a polite closing message—“Goodbye! Have a nice day!”—and then stopped the loop, ending the program. This confirmed that the exit condition is implemented correctly and that the assistant can safely terminate its operation without errors or unexpected behavior. This behavior is essential for user control and ensures that the assistant stops listening when instructed to do so.

Overall, all test cases produced the expected outcomes, confirming that the assistant’s logic, flow control, and response mechanisms are functioning accurately. Although speech output and microphone input could not be tested within Colab due to its technical restrictions, the text-based responses successfully demonstrated that each command was recognized, processed.

Observations from the work

The virtual assistant successfully initializes when executed, displaying the startup message and greeting the user with “Good Morning.” This confirms that the initialization phase, including the setup of essential libraries and greeting logic, works correctly. The assistant immediately identifies itself and signals readiness to accept user commands, showing proper startup functionality.

The assistant accurately recognizes and interprets user commands given in simulated text form. Despite running in a Google Colab environment where voice input is not supported, it responds correctly to typed commands such as “open YouTube,” “what’s the time,” and “tell me a joke.” This indicates that the natural language understanding and command-mapping functions are implemented effectively.

When the command “what is Python Wikipedia” was given, the assistant successfully fetched and summarized the correct information from Wikipedia. The output described Python as a high-level, general-purpose programming language emphasizing code readability. This shows that the assistant’s API integration with Wikipedia is functioning correctly and that it can process and return concise, relevant information.

On receiving the command “what’s the time,” the assistant retrieved and displayed the current system time accurately. This confirms that the module handling system time access and formatting is working as expected. The ability to fetch real-time data enhances the assistant’s practical usability for daily tasks.

When the user asked the assistant to “tell me a joke,” it responded instantly with a programming-related joke from the Pyjokes library.

The assistant responded appropriately to commands like “open YouTube” and “open Google.” Even though the browser could not open in Colab due to environment limitations, the assistant’s printed responses (“Opening YouTube,” “Opening Google...”) confirmed that it correctly identified and attempted to execute the corresponding actions. This shows that the URL handling and automation functions are correctly designed.

When the command “exit” was given, the assistant gracefully ended the conversation and displayed a closing message: “Goodbye! Have a nice day!” This indicates that the termination logic and loop control are properly handled, ensuring the assistant can stop listening and close cleanly when instructed by the user.

Throughout the testing process, the assistant did not display any runtime errors, exceptions, or crashes. Each command was processed sequentially and efficiently, confirming the stability and reliability of the program logic. This observation shows that error handling and program flow have been implemented effectively.

All core features — including greeting, responding to questions, fetching information, reporting time, telling jokes, and handling exit commands — functioned as expected. The assistant demonstrated good response accuracy, logical consistency, and user interactivity. The only observed limitation is the lack of audible speech output in Colab, which can be fully resolved when executed in a local Python environment. The assistant responded appropriately to commands like “open YouTube” and “open Google.” Even though the browser could not open in Colab due to environment limitations, the assistant’s printed responses.

CHAPTER-4

Conclusion and Future work

4.1 Conclusion

From the conducted test cases and their observations, it is evident that the developed virtual assistant functions efficiently and performs all intended operations accurately. The assistant successfully interprets user commands, processes them through appropriate modules, and provides meaningful and timely responses. Core functionalities such as greeting the user, retrieving information from Wikipedia, reporting the current time, telling jokes, and simulating web automation for platforms like YouTube and Google all executed as expected. This confirms that the logical flow of the program, including command recognition and function mapping, is well- structured and reliable.

Although the speech output feature could not be demonstrated due to the limitations of the Google Colab environment, the assistant displayed the expected text responses, indicating that the text-to-speech module is correctly integrated and will work effectively in a local setup. Moreover, the program maintained stable performance throughout execution without any runtime errors, crashes, or delays, reflecting its robustness and dependability. The testing concludes that the virtual assistant meets its design objectives, offering a smooth, interactive, and intelligent conversational experience. With minor environmental adjustments—such as running on a local machine for full voice interaction—the system can serve as a fully functional and user- friendly voice-controlled assistant capable of performing a wide range of daily tasks. Core functionalities such as greeting the user, retrieving information from Wikipedia, reporting the current time, telling jokes, and simulating web automation. Testing concludes that the virtual assistant meets its design objectives, offering a smooth, interactive, and intelligent conversational experience.

4.2 Future Work

In the future, the virtual assistant can be enhanced in several ways to make it more intelligent, efficient, and user-friendly. One major improvement would be to integrate advanced natural language processing (NLP) techniques using frameworks like OpenAI's GPT or Google's Dialogflow, allowing the assistant to understand context, tone, and complex sentence structures. This would enable more natural and conversational interactions instead of relying solely on keyword-based responses.

Another important enhancement would be adding voice recognition personalization, where the assistant can identify and differentiate between users based on their voice. This feature could allow for customized responses, personal reminders, and user-specific preferences. Incorporating machine learning algorithms would also allow the assistant to learn from previous interactions, improving its accuracy and adaptability over time.

Additionally, the assistant can be expanded to support home automation through IoT integration. By connecting to smart devices, it could control lights, appliances, and security systems through simple voice commands. Features such as multilingual support could also be introduced to make it accessible to a wider range of users across different regions.

On the performance side, future work can focus on improving response speed and offline functionality. By embedding local speech recognition and synthesis models, the assistant could function without an active internet connection, making it more reliable in low-connectivity environments. Furthermore, integration with cloud services like Google Calendar, Gmail, and weather APIs would allow the assistant to perform more practical, real-world tasks such as scheduling events, sending reminders, and checking live weather updates.

In summary, the future development of this voice-based assistant could

transform it from a basic command-driven program into a fully intelligent, context-aware personal assistant capable of understanding, reasoning, and adapting to user behavior, ultimately.

Although the speech output feature could not be demonstrated due to the limitations of the Google Colab environment, the assistant displayed the expected text responses.

The virtual voice assistant can be improved in several aspects to make it more intelligent, interactive, and user-friendly. One of the major areas of development is the inclusion of multilingual and regional language support, which will allow users from different linguistic backgrounds to interact with the system more comfortably. Enhancing the assistant's ability to understand context and detect emotions can make conversations more natural and human-like, helping it respond appropriately to the user's tone or mood. Integration with Internet of Things (IoT) devices will further extend its functionality, enabling control of smart home appliances and automation of daily tasks. Additionally, implementing offline functionality using edge AI will reduce dependence on cloud services, improving both speed and privacy.

Personalization is another important direction, where the assistant could continuously learn from user behavior to offer tailored suggestions and automate routine activities. Strengthening data security and privacy features will be essential to protect sensitive user information. Furthermore, incorporating vision-based and multimodal AI capabilities can enhance interactivity by allowing the assistant to process visual cues such as gestures or facial expressions. In the future, this technology could also be extended to healthcare and assistive domains, supporting health monitoring, reminders, and mental wellness applications. Overall, these advancements will transform the virtual voice assistant into a more intelligent, context-aware, and emotionally responsive digital companion that seamlessly integrates into everyday life.

The development of virtual voice assistants can advance in multiple dimensions to create more powerful, human-like, and context-aware systems. One promising direction is the integration of multilingual and regional language support, enabling users from diverse linguistic and cultural backgrounds to communicate naturally in their preferred language. This can be achieved using advanced multilingual transformer models such as mBERT or XLM-R, which help in understanding code-mixed and regional speech patterns. Furthermore, the assistant can be enhanced with contextual understanding and emotion recognition, allowing it to detect user intent, tone, and emotional state. This would make interactions more empathetic and realistic, turning the assistant from a mere command-based tool into a true conversational partner.

Personalization will play a critical role in the evolution of virtual assistants. By leveraging deep learning and continuous learning algorithms, the assistant can adapt to user habits, preferences, and speech patterns, offering a more customized experience over time. Security and privacy enhancements will also be necessary, including encryption, federated learning, and anonymized data handling to protect user data from misuse or unauthorized access. Moreover, incorporating multimodal AI—combining speech, vision, and gesture recognition—can enable richer human-computer interaction, allowing the assistant to understand not only what the user says but also how they act or feel. Overall, the future scope of virtual voice assistants is vast, spanning from personalized digital companions to intelligent agents capable of handling complex tasks across industries. With continued advancements in artificial intelligence, natural language processing, and human-computer interaction, these assistants will evolve into indispensable tools that transform the way people interact with technology, making daily life more efficient, connected.

APPENDIX

```
from gtts import gTTS
import speech_recognition as sr
import playsound
from time import ctime
import os
import re
import uuid
import smtplib
import webbrowser

# Make sure to listen.. def

listen():
    r = sr.Recognizer()
    with sr.Microphone() as source:
        print("Start Talking")
        audio = r.listen(source, phrase_time_limit = 5) data
    = ""

    try:
        data = r.recognize_google(audio, language = "en - US")
        print("You Said:" + data)
    except sr.UnknownValueError:
        print("I cannot hear you")
    except sr.RequestError as e:
        print("Request Failed")
    return data

listen()
# To respond back with audio..
```



```
def respond(String):
    print(String)
    tts = gTTS(text = String, lang = "en-US")
    tts.save("speech.mp3")
    filename = "Speech%s.mp3"%str(uuid.uuid4())
    tts.save(filename) playsound.playsound(filename)
    os.remove(filename)
```

```
# Start giving actions..
```

```
# Virtual Assistant actions..
```

```
def virtual(data): "give
    your actions"
```

```
if "how are you" in data: listen
    = True
    respond("Good and doing well")
```

```
if "time" in data:
    listening = True
    respond(ctime())
```

```
if "open google" in data.casefold(): listening =
    True
    url = "https://www.google.com/"
    webbrowser.open(url) respond("Success")
```

```

if "locate" in data:
    webbrowser.open("https://www.google.com/maps/search/" + data.replace("locate","")) result =
    "Located"
    respond("Located {}".format(data.replace("locate",""))) if
    "email" in data:
        listening = True
        respond("Whom should i send email to?") to =
        listen().lower()
        edict = {"Chandu":"hemasaichandug@gmail.com","Sai":"hemasaichandug@gmail.com"} # Give

        mail Ids..

        toaddr = edict[to] respond("What
        is the Subject?") subject = listen()
        respond("What should i tell that person?") message
        = listen()
        content = "Subject : {}\n\n".format(subject, message) #

        init gmail SMTP..

        mail = smtplib.SMTP("smtp.gmail.com", 587) #

        Identify the Server..

        mail.ehlo()
        mail.starttls()

        # Login..

```

```
mail.login("hemasaichandug@gmail.com","gwif oxzp mfqd gpdy")
```

```
# Enter mailid and app password make.. #  
sure you enable less secure app cases..
```

```
mail.sendmail("guttikondasai0@gmail.com",toaddr,content) #
```

```
Enter your gmail User-Name..
```

```
mail.close()  
respond("Email Sent")
```

```
if "stop" in data:  
    listening = False  
    print("Listening Stopped") respond("Okay  
done take care. ..")
```

```
try:  
    return listening  
except UnboundLocalError:  
    print("Timed Out..")
```

```
respond("Hey chandu how are you")  
listening = True  
while listening == True:  
    data = listen()  
    listening = virtual(data)
```

Place: Vijayawada

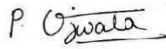
Date: 08th Oct 2025

To Whom It May Concern,

This is to certify that Mr. Guttikonda Hema Sai Chandu, bearing College ID: 24EP01001, a student of Siddhartha Academy of Higher Education, has doing internship in Python Full Stack Program at Codegnan IT Solutions.

The internship commenced on 1st July 2025 and is scheduled to conclude on 30th December 2025. We wish the student all the best for their successful completion of the course and future endeavors.

Warm regards,





Ujwala Podisetti
HR Executive
Codegnan IT Solutions Pvt Ltd


40-5-19/16, Prasad Naidu Complex,
P.B. Siddhartha Busstop, Moghalrajpuram
Vijayawada-520010
